

System Design Document: Fakturama Image-to-Cash Automation

Candidate: Mohanad Emad

Email: mohanad200130@gmail.com

Phone: +201144111251

1. Executive Summary

The objective of this system is to seamlessly convert a single order image into a fully persisted Order and linked Invoice within the Fakturama desktop application. To achieve high resilience and avoid the pitfalls of brittle, coordinate-based automation, the system employs a decoupled, state-driven architecture. This architecture separates unstructured data processing (via an LLM) from strict desktop UI interactions (via UIA) and orchestrates them through a deterministic Finite State Machine (FSM).

2. Image-Extraction & Data Normalization Strategy

The pipeline completely bypasses traditional OCR templates, which often fail under minor layout variations. Instead, it utilizes a multimodal LLM (Google Gemini) operating with strict structured outputs to ensure extraction fidelity.

2.1. Multimodal Parsing with Enforced Schema

- **Direct Image Consumption:** The raw order image is passed directly to the vision-capable LLM to handle spatial reasoning and complex nested tables without intermediate bounding-box mapping.
- **Pydantic Data Validation:** The extraction layer defines a comprehensive Pydantic schema mapping to the exact required Fakturama entities (Order, Debtor, Address, Product, Line Item). This schema is passed into the GenAI SDK's `response_schema` parameter.
- **In-Flight Normalization:** The LLM is prompted to perform the business logic required by Fakturama natively, such as standardizing "Bank Transfer" to the system's expected terminology and parsing single-line addresses into Street, ZIP, and City variables.

2.2. Deterministic Calculation Boundaries

While the LLM excels at text extraction, floating-point arithmetic is explicitly moved to the Python validation layer. The calculation for the Product Master Gross Price ($\text{Unit_Net_Price} * (1 + \text{VAT_Percentage} / 100)$) is executed within a Pydantic `@model_validator`, ensuring deterministic rounding to two decimal places and absolute precision before UI execution begins.

3. Control-Discovery & Grounding Strategy

The system interacts with Fakturama via the native Microsoft UI Automation (UIA) API using the `uiautomation` Python wrapper. This approach guarantees zero reliance on screen coordinates or display resolutions.

3.1. Navigating Eclipse RCP Complexity

- **Restricted Search Depth:** Because Fakturama is built on the Eclipse Rich Client Platform, its UI tree is exceptionally deep and heavily nested. To prevent the automation from freezing during recursive searches, the pipeline utilizes strict `searchDepth` constraints when locating child nodes from known parent windows.
- **Dynamic Locators:** Elements are targeted via robust combinations of Name, `ControlType`, and spatial relationships (e.g., locating a specific ``Edit`` field by finding the ``Text`` label immediately preceding it in the accessibility tree).

3.2. Asynchronous State Polling

To account for application rendering latency (e.g., waiting for the "New Product" modal to appear), the system heavily utilizes explicit `WaitForExistence()` calls. It polls the UI tree for the desired window handle or control state rather than relying on arbitrary sleep durations.

4. Orchestration & State Management

The procedural logic is governed by `python-statemachine`. This enforces a strict chronological flow and safely manages the context-switching required when missing Master Data disrupts the Order creation process.

Current State	Trigger Action	Target State / Outcome
IMAGE_EXTRACTION	Pydantic validation succeeds	ORDER_INITIALIZATION
DEBTOR_SEARCH	Exact match found in UIA Grid	PRODUCT_SEARCH
DEBTOR_SEARCH	No match found; Context switch	DEBTOR_CREATION
PRODUCT_SEARCH	SKU missing	VAT_VERIFICATION → PRODUCT_CREATION
ORDER_VERIFICATION	Line items & Totals matched	INVOICE_GENERATION

5. Architectural Tradeoffs & Mitigations

- **LLM API Latency vs. OCR Speed:** Passing images to an external API introduces network latency (typically 5–15 seconds). However, this tradeoff is acceptable as it effectively guarantees normalized, structured text, drastically reducing the engineering overhead of maintaining localized regex rules for dynamic invoices.
- **State Machine Overhead vs. Procedural Scripting:** Implementing a declarative state machine takes more upfront setup time than writing nested if/else statements. The benefit, however, is a highly readable, self-documenting orchestration layer that prevents infinite loops if a specific window fails to open or close during complex context switching.
- **Low-Level Locators:** Using uiautomation requires deep inspection of the Fakturama UI tree using tools like Accessibility Insights. While maintenance of these locators is higher than relying on surface-level image matching, the execution is virtually immune to screen resolution changes, DPI scaling issues, or overlapping windows.

6. Written Questionnaire: Future Scope

If I had 3 more hours, what would I do for this task?

- **Implement Multi-Page Table Stitching:** Enhance the Pydantic schema and LLM prompt to dynamically handle multi-page invoices with carry-over totals, ensuring no line items are dropped during pagination.

- **Enhanced Error Recovery:** Add robust fallback mechanisms in the FSM to gracefully tear down the active Order if a critical creation step (like a duplicate Payment Term conflict) occurs, resetting the Fakturama environment to a clean state.
- **Comprehensive Unit Testing:** Mock the UIA node responses using Python's `unittest.mock` to validate the state transitions and math calculations entirely offline before execution against the live desktop application.